

Pembuatan Aplikasi Chat Web dengan Kriptografi Kunci-Simetri dan Kunci-Publik

II4021 Kriptografi



Disusun oleh:

Willhelmina Rachel Silalahi / 18222049

Nathaniel Liady / 182221114

Daniel A. M. Sipayung / 18223031

Program Studi Sistem dan Teknologi Informasi
Sekolah Teknik Elektro dan Informatika - Institut Teknologi Bandung
Jl. Ganesha 10, Bandung 40132

	Program Studi Sistem Teknologi Informasi STEI – ITB	dan	Nomor Dokumen	Jumlah Halaman
			T-02/II4021-49_8 9_114	39

Daftar Isi

Daftar Isi	2
Daftar Gambar	3
Pernyataan	5
Dasar Teori	6
A. JSON Web Token (JWT)	6
B. Elliptic Curve Diffie-Hellman (ECDH)	7
C. Fungsi Hash	7
D. Advanced Encryption Standard (AES)	7
Perancangan dan Implementasi	8
A. Infrastruktur Teknologi	8
B. Skema Basis Data	8
C. Implementasi per Tahap	9
1. Registrasi Pengguna	10
2. Login	10
3. Library JWT	11
4. Daftar Kontak	11
5. Pembentukan Kunci Komunikasi	12
6. Pengiriman Pesan	12
7. Penerimaan dan Dekripsi Pesan	13
8. Bonus 1 : Integritas dan Autentikasi Pesan	13
9. Bonus 2 : Unit Test Library JWT	13
10. Bonus 3 : Docker	13
Pengujian dan Analisis	15
A. Uji autentikasi dan manajemen pengguna	15
1. Registrasi dengan data valid	15
2. Registrasi dengan email terdaftar	16
3. Login dengan password yang benar	17
4. Login dengan password yang salah	18
B. Uji implementasi JSON Web Token (JWT)	19
1. Verify dengan Public Key Benar	19
2. Verify dengan public key yang salah	21
3. Uji token dengan format tidak valid	22
4. Uji token dengan klaim waktu (exp/nbf) yang tidak sesuai	23
C. Uji pembentukan kunci komunikasi ECDH dan KDF	26
1. Dua pengguna melakukan key exchange	26
2. Shared secret berhasil menjadi kunci simetris	26
D. Uji enkripsi dan dekripsi pesan:	28
1. Kirim pesan dengan kunci yang benar	28

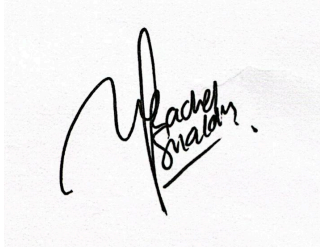
2. Kirim pesan dengan kunci yang salah atau tidak sesuai	30
E. Uji integritas dan autentikasi pesan (MAC):	32
1. Kirim pesan dengan MAC yang valid	32
2. Ubah sebagian isi pesan atau MAC	34
Kesimpulan	37
Daftar Pustaka	38
Lampiran	39
Pranala Repositori	39
Pranala Video Demo	39
Pembagian Tugas	39

Daftar Gambar

Gambar 1. Registrasi Kripto Chat	16
Gambar 2. Data disimpan pada file JSON	17
Gambar 3. Registrasi menggunakan email yang telah terdaftar	18
Gambar 4. Login menggunakan password yang benar	19
Gambar 5. JWT tersimpan pada localStorage	19
Gambar 6. Login menggunakan password yang salah	20
Gambar 7. JWT hasil login yang tersimpan pada localStorage	21
Gambar 8. Payload JWT berhasil dibaca melalui browser console	21
Gambar 9. Tampilan pesan dengan public key yang benar	22
Gambar 10. Hasil verifikasi JWT gagal dengan public key yang salah	23
Gambar 11. Tampilan pesan saat verifikasi JWT gagal	23
Gambar 12. Hasil verifikasi JWT gagal dengan format tidak valid	24
Gambar 13. Tampilan pesan saat format token invalid	24
Gambar 14. Hasil verifikasi JWT gagal dengan exp yang kedaluwarsa	25
Gambar 15. Tampilan pesan saat exp yang kedaluwarsa	25
Gambar 16. Hasil verifikasi JWT gagal dengan nbf belum aktif	26
Gambar 17. Tampilan pesan saat nbf belum aktif	26
Gambar 18. Shared Secret pada kedua pengguna	27
Gambar 19. Pesan hasil dari enkripsi dan dekripsi	28
Gambar 20. Pesan terenkripsi disimpan pada basis data	28
Gambar 21. Alice mengirim pesan kepada Bob	30
Gambar 22. Pesan Alice kepada Bob dienkripsi pada basis data	30
Gambar 23. Pesan Alice terkirimkan kepada Bob	30
Gambar 24. Bob mengirim pesan kepada Alice	31
Gambar 25. Pesan Bob yang telah dienkripsi	32
Gambar 26. Modifikasi pada cipherteks pesan Bob	33
Gambar 27. Cipherteks yang dimodifikasi tidak dapat didekripsi	33
Gambar 28. Pesan Alice kepada Bob	34
Gambar 29. MAC yang valid	34
Gambar 30. Pesan Alice terkirim kepada Bob	35
Gambar 31. Pesan Bob kepada Alice	36
Gambar 32. MAC pesan yang valid dari bob	36
Gambar 33. Modifikasi MAC dari pesan	37
Gambar 34. Pesan Bob yang diterima Alice menggunakan MAC yang tidak valid	37

Pernyataan

Kami menyatakan bahwa kode program yang dihasilkan bukan merupakan hasil salinan mentah (*raw output*) dari Generative AI, melainkan hasil pengembangan dan penulisan mandiri.

A handwritten signature in black ink on a light background. The signature is stylized, with the first letter 'W' being large and prominent. The name 'Rachel Silalahi' is written in a cursive script.

Willhelmina Rachel Silalahi

A handwritten signature in black ink. The signature is written in a cursive script, with the letters 'n', 'i', and 'e' being prominent.

Nathaniel Liady

A handwritten signature in black ink. The signature is written in a cursive script, with the letters 'D', 'A', and 'M' being prominent.

Daniel A. M. Sipayung

Dasar Teori

A. JSON Web Token (JWT)

JSON Web Token merupakan *open standard* (RFC 7519) untuk mengirimkan informasi dari dua belah pihak (umumnya antara *client* dan *server*) secara aman. Setiap JWT akan ditandatangani secara digital menggunakan kriptografi asimetris untuk mencegah adanya perubahan informasi tentang pengguna atau sesi.

Struktur JWT tersusun atas tiga bagian yang dipisahkan oleh titik (.) yaitu *header*, *payload*, dan *signature*. *Header* mendefinisikan tipe JWT dan algoritma yang digunakan untuk menandatangani token. *Payload* berisi informasi seperti ID pengguna, waktu kadaluarsa, dan peran. *Signature* memastikan integritas JWT dengan menandatangani *header* dan *payload* dengan menggunakan *private key* yang dapat diverifikasi menggunakan *public key*. JWT juga mengikuti standar konvensi (RFC 7519) yang terdiri dari *iss* (*issuer* yang membuat token), *aud* (*audience* aplikasi penerima yang dituju), *sub* (*subject* yang mengidentifikasi pengguna), *exp* (*expiration time* yaitu waktu kadaluarsa), *iat* (*issued at time* yaitu waktu diterbitkannya JWT), *nbf* (*not before* yaitu waktu mulai yang sesuai), dan *email*, *email_verified*, *roles* yang disesuaikan sesuai kebutuhan aplikasi.

Sebagai contoh, Google membuat JWT berikut saat *sign in*

```
{
  "iss": "https://accounts.google.com",
  "aud": "12345678900.apps.googleusercontent.com",
  "sub": "12345566767",
  "email": "kriptografi@example.com",
  "email_verified": true,
  "iat": 1353601026,
  "exp": 1353604926
}
```

JWT ini digunakan oleh aplikasi untuk memverifikasi pengguna saat token dibuat dan menentukan apakah token tersebut masih berlaku.

JWT digunakan oleh *server* untuk mengautentikasi pengguna dan memungkinkan verifikasi lokal tanpa memerlukan pemeriksaan ke basis data yang meningkatkan performa autentifikasi. Saat pengguna *login* dengan data *email* dan *password* yang sesuai, *server* akan membuat JWT yang akan ditandatangani dengan kriptografi *Elliptic Curve Digital Signature Algorithm*. Kemudian *client* akan menyimpan token tersebut secara lokal. Saat *client* ingin melakukan sesuatu yang memerlukan verifikasi, *client* memasukkan JWT pada *header*. Akhirnya, *server* menggunakan *public key* untuk memeriksa kebenaran tanda tangan dan kesesuaian masa berlaku token.

B. Elliptic Curve Diffie-Hellman (ECDH)

Algoritma ini merupakan sistem kriptografi kurva eliptik yang diadopsi dari algoritma Diffie-Hellman. Algoritma Diffie-Hellman merupakan algoritma yang memungkinkan dua pihak untuk menyepakati kunci rahasia bersama untuk mengenkripsi pesan pada saat mereka berkomunikasi. Sebagai contoh Alice dan Bob yang bersepakat atas parameter publik bilangan prima p dan *generator* g . Kemudian masing-masing dari mereka membuat kunci privat dan kunci publiknya sendiri. Kemudian mereka bertukar kunci publik dan dapat menentukan kunci rahasia yang ditentukan dari kunci publik pihak lain dan kunci privatnya sendiri.

Elliptic Curve Diffie-Hellman bekerja berdasarkan prinsip Algoritma Diffie-Hellman yang dilakukan dengan kurva eliptik. Alice dan Bob sepakat atas jenis kurva dan titik acuan dan memilih bilangan acak sebagai kunci privat serta menurunkan kunci publiknya. Setelahnya mereka dapat menghitung *shared secret* yang identik untuk berkomunikasi.

Pada implementasi aplikasi tugas ini, ECDH digunakan untuk mendapatkan *shared secret* antara dua pengguna untuk dapat berkomunikasi. Penggunaan ECDH pada *chat app* digunakan karena kelebihan dari *Elliptic Curve Cryptography* yaitu ukuran kunci yang dihasilkan lebih kecil dengan tingkat keamanan yang setara.

C. Fungsi Hash

Fungsi *hash* mengubah data berukuran sembarang menjadi nilai *hash* berukuran tetap. Sifat dari fungsi *hash* kriptografis ialah sebagai berikut :

- *Collision resistance*: kesulitan menemukan dua input berbeda yang menghasilkan nilai hash yang sama
- *Pre-image resistance*: sukar untuk menemukan input a sedemikian sehingga nilai *hash* dari a adalah y sembarang.
- *Second pre-image resistance*: Untuk a dan b dengan ketentuan $H(a) = b$, sukar untuk menemukan nilai a' yang bila dimasukkan dalam fungsi H , $H(a')$, akan menghasilkan nilai b yang sama.

Perubahan sekecil apapun pada masukan data akan menghasilkan nilai *hash* yang berbeda. Pada tugas ini, *hash* dipakai untuk melindungi *password* pengguna sebelum nantinya akan dikombinasikan dengan *salt* untuk disimpan ke basis data.

D. Advanced Encryption Standard (AES)

Advanced Encryption Standard merupakan algoritma enkripsi simetris yang bekerja pada blok data dengan ukuran 128 bit dengan panjang kunci 128, 192, atau 256 bit. Pada tugas ini digunakan AES-256 CBC (*Cipher Block Chaining*). Mode ini mengenkripsi setiap balok plainteks dengan melakukan operasi XOR terlebih dahulu kepada blok cipher teks sebelumnya sebelum dienkripsi, maka setiap blok keluaran bergantung dengan blok sebelumnya.

Perancangan dan Implementasi

A. Infrastruktur Teknologi

Berikut adalah infrastruktur teknologi (*tech stack*) yang kami gunakan untuk mengimplementasikan secure web chat application dengan autentikasi JWT, pertukaran kunci menggunakan ECDH, enkripsi pesan menggunakan AES-256-GCM, serta autentikasi dan integritas pesan menggunakan HMAC. Pemilihan teknologi ini dilakukan dengan mempertimbangkan kesederhanaan arsitektur aplikasi, kemudahan implementasi, serta kesesuaian dengan spesifikasi tugas.

Tabel 2.1 Pemilihan infrastruktur teknologi

Komponen	Teknologi
Front End	HTML, CSS, JavaScript
Back End	Node.js native HTTP server
Basis Data / Storage	JSON file (data/db.json)
Kriptografi Client	Web Crypto API
JWT Library	Custom library (src/jwt.js)
Testing	node:test
Deployment	Docker dan Docker Compose

B. Skema Basis Data

Aplikasi menggunakan penyimpanan berbasis berkas JSON lokal. Ketika server pertama kali dijalankan, folder data dibuat secara otomatis. Jika file `data/db.json` belum tersedia, server menginisialisasi struktur awal berisi array users dan messages. Server juga membangkitkan pasangan kunci ECDSA untuk penandatanganan JWT dan menyimpannya sebagai `data/jwt-private.pem` serta `data/jwt-public.pem`. Berikut tabel-tabel yang terlibat dalam basis data beserta atributnya.

Tabel 2.2 Tabel users

Nama tabel	users	
Deskripsi	Tabel yang menyimpan data akun pengguna.	
Parameter	Tipe Data	Keterangan
Id (PK)	string	ID unik pengguna yang dibuat menggunakan <code>crypto.randomUUID()</code> .
email	string	Email pengguna yang telah dinormalisasi menjadi huruf kecil. Email digunakan sebagai identitas login.
passwordSalt	string	Salt yang digunakan dalam proses hashing password.

passwordHash	string	Hasil hashing password menggunakan crypto.scryptSync(), disimpan dalam format Base64.
publicKeyJwk	object	Public key ECDH pengguna dalam format JWK. Public key ini digunakan pengguna lain untuk melakukan key exchange.
privateKeyEnvelope	object	Private key ECDH pengguna yang sudah dienkripsi di sisi client sebelum dikirim ke server.
createdAt	string	Waktu pembuatan akun dalam format ISO string.

Tabel 2.3 Tabel messages

Nama tabel	messages	
Deskripsi	Tabel yang menyimpan data pesan antar pengguna.	
Parameter	Tipe Data	Keterangan
id (PK)	string	ID unik pesan yang dibuat menggunakan crypto.randomUUID().
senderEmail (FK)	string	Email pengguna yang mengirim pesan. Mengacu pada users.email.
receiverEmail (FK)	string	Email pengguna yang menerima pesan. Mengacu pada users.email.
ciphertext	string	Isi pesan yang sudah dienkripsi menggunakan AES-256-GCM.
iv	string	Initialization vector yang digunakan pada proses enkripsi pesan.
mac	string	Nilai HMAC untuk memverifikasi integritas dan autentikasi pesan.
timestamp	string	Waktu pesan dibuat/dikirim dalam format ISO string.
alg	string	Informasi algoritma yang digunakan, yaitu AES-256-GCM dan HMAC-SHA-256.

C. Implementasi per Tahap

Dapat diperhatikan bahwa aplikasi berbentuk *web app* dengan pemrosesan pada sisi klien dan sisi server. Pemrosesan pada sisi klien digunakan untuk membangkitkan kunci, mengenkripsi private key, membentuk kunci komunikasi,

mengenkripsi pesan, serta mendekripsi pesan. Sementara itu, sisi server digunakan untuk **menyediakan endpoint API, memvalidasi JWT, menyimpan data pengguna, dan menyimpan pesan terenkripsi.**

Perlu diingat bahwa endpoint yang mengakses data kontak dan pesan mengharuskan adanya autentikasi pengguna melalui JWT. Implementasi utama pada **sisi klien** terdapat pada **public/app.js**, sedangkan implementasi utama pada **sisi server** terdapat pada **server.js**. Library JWT custom diimplementasikan pada `src/jwt.js`.

1. Registrasi Pengguna

Pada tahap ini, pengguna mengisi email dan password melalui antarmuka. Sebelum data dikirim ke server, **pasangan kunci ECDH dibangkitkan di sisi klien menggunakan Web Crypto API dengan kurva P-256**. Public key diekspor dalam format JWK, sedangkan private key dienkripsi terlebih dahulu sebelum dikirim ke server.

Private key dienkripsi menggunakan AES-256-GCM. Kunci enkripsi private key diturunkan dari password pengguna menggunakan **PBKDF2 dengan SHA-256, salt acak, dan 150.000 iterasi**. Hasil enkripsi private key disimpan dalam bentuk `privateKeyEnvelope`.

Tabel 2.4 Data yang dikirim ke server untuk registrasi pengguna

Parameter	Tipe Data	Keterangan
email	string	Email pengguna baru.
password	string	Password pengguna baru.
publicKeyJwk	object	Public key ECDH pengguna dalam format JWK.
privateKeyEnvelope	object	Private key ECDH yang telah dienkripsi di sisi klien.

Endpoint `/api/register` menerima data tersebut. Server kemudian memeriksa apakah email sudah terdaftar. Jika belum, password akan di-*hash* menggunakan *scrypt* dengan salt unik. Data pengguna yang disimpan meliputi email, hash password, salt, public key, private key terenkripsi, dan waktu pembuatan akun.

2. Login

Setelah membuat akun, pengguna perlu login dengan mengisi email dan password pada formulir login. Data tersebut dikirim ke endpoint `/api/login` untuk divalidasi oleh server. Server akan mencari akun berdasarkan email. Password yang dimasukkan pengguna di-*hash* kembali menggunakan salt yang tersimpan,

kemudian hasilnya dibandingkan dengan *passwordHash* di basis data. Jika sesuai, server akan menerbitkan JWT menggunakan fungsi sign dari library *src/jwt.js*.

JWT yang diterbitkan menggunakan algoritma ES256 dan berisi klaim seperti iss, sub, aud, iat, nbf, exp, dan jti. Token ini dikirim ke client dan disimpan di *localStorage*. Selain itu, client juga menerima *privateKeyEnvelope*, lalu mendekripsinya menggunakan password login agar private key dapat digunakan untuk proses ECDH.

3. Library JWT

Library JWT diimplementasikan pada file *src/jwt.js*. Library ini memiliki dua fungsi utama, yaitu **sign** dan **verify**.

Fungsi sign digunakan untuk membuat JWT dengan format:

<code>base64url(header).base64url(payload).base64url(signature)</code>
--

Fungsi ini menerima **header, payload, claims, dan private key**. Header harus berisi tipe JWT dan algoritma yang digunakan. Algoritma yang didukung adalah ES256, ES384, dan ES512.

Fungsi verify digunakan untuk memeriksa JWT yang diterima server. Pemeriksaan meliputi format token, algoritma, signature, serta klaim seperti exp, nbf, iss, aud, sub, dan jti. Jika token tidak valid, fungsi akan melempar error sehingga request ditolak.

4. Daftar Kontak

Setelah berhasil login, pengguna dapat melihat daftar kontak. Daftar kontak merupakan seluruh pengguna yang sudah pernah mendaftar, kecuali pengguna yang sedang login.

Endpoint **/api/contacts** akan memverifikasi JWT terlebih dahulu. Jika token valid, server mengembalikan daftar kontak dalam bentuk email dan public key.

Tabel 2.5 Data kontak yang dikirim ke client

Parameter	Tipe Data	Keterangan
email	string	Email pengguna lain.
publicKeyJwk	object	Public key ECDH milik kontak.

Public key kontak dikirim karena diperlukan untuk membentuk **shared secret pada proses key exchange**. Data sensitif seperti password hash, salt, dan private key tidak dikirim ke client lain.

5. Pembentukan Kunci Komunikasi

Saat pengguna memilih salah satu kontak, client akan menjalankan proses pembentukan kunci komunikasi. Public key kontak yang diterima dari server diimpor kembali menggunakan **Web Crypto API**. Setelah itu, client menghitung shared secret menggunakan private key miliknya sendiri dan public key milik kontak.

Shared secret hasil ECDH tidak langsung digunakan sebagai kunci enkripsi. Nilai tersebut diproses menggunakan **HKDF dengan SHA-256**. Dari proses ini, sistem menghasilkan dua kunci berbeda, yaitu **kunci AES untuk enkripsi pesan dan kunci HMAC untuk autentikasi pesan**.

Dengan cara ini, kedua pengguna yang berkomunikasi dapat memperoleh kunci yang sama. Server hanya menyimpan dan mengirim public key, tetapi tidak mengetahui shared secret maupun kunci AES yang digunakan untuk membaca pesan.

6. Pengiriman Pesan

Pesan yang ingin dikirimkan oleh pengguna diproses terlebih dahulu di sisi klien. Pesan tidak dikirim dalam bentuk plainteks. Client membuat IV acak, kemudian mengenkripsi pesan menggunakan AES-256-GCM dengan kunci AES hasil HKDF.

Setelah pesan dienkripsi, client menghitung HMAC-SHA-256 dari metadata pesan dan ciphertext. MAC ini digunakan untuk memastikan bahwa pesan tidak berubah selama pengiriman atau penyimpanan.

Tabel 2.6 Data yang dikirim ke server untuk pengiriman pesan

Parameter	Tipe Data	Keterangan
receiverEmail	string	Email penerima pesan.
ciphertext	string	Pesan yang sudah dienkripsi.
iv	string	IV yang digunakan pada AES-GCM.
mac	string	HMAC-SHA-256 untuk autentikasi pesan.
timestamp	string	Waktu pengiriman pesan.
alg	string	Informasi algoritma yang digunakan.

Endpoint **/api/messages** dengan metode POST menerima data tersebut. Server memvalidasi JWT, memeriksa penerima, lalu menyimpan pesan ke basis data. Server tidak melakukan dekripsi dan tidak mengetahui isi plainteks pesan.

7. Penerimaan dan Dekripsi Pesan

Saat pengguna membuka chatroom, riwayat pesan diperoleh melalui endpoint **/api/messages?with=email_kontak**. Endpoint ini mengembalikan pesan dua arah antara pengguna yang sedang login dan kontak yang dipilih. Client melakukan polling secara berkala agar pesan baru dapat ditampilkan.

Setelah pesan diterima, client terlebih dahulu memverifikasi MAC. Jika MAC tidak valid, pesan ditandai sebagai pesan yang tidak sah. Jika MAC valid, client akan mendekripsi ciphertext menggunakan AES-256-GCM dan kunci AES percakapan. Apabila dekripsi berhasil, pesan ditampilkan sebagai plainteks pada antarmuka. Jika dekripsi gagal, aplikasi menampilkan penanda bahwa pesan tidak dapat didekripsi.

8. Bonus 1 : Integritas dan Autentikasi Pesan

Bonus integritas dan autentikasi pesan diimplementasikan menggunakan HMAC-SHA-256. MAC dihitung di sisi klien dari metadata pesan dan ciphertext, kemudian dikirim bersama payload pesan.

Pada sisi penerima, MAC diverifikasi terlebih dahulu sebelum proses dekripsi dilakukan. Jika nilai MAC tidak sesuai, pesan tidak didekripsi dan ditandai sebagai tidak valid. Dengan demikian, sistem dapat mendeteksi perubahan pada pesan sebelum pesan diproses lebih lanjut.

9. Bonus 2 : Unit Test Library JWT

Unit test untuk library JWT dibuat pada file **test/jwt.test.js** dan dijalankan menggunakan node:test. Pengujian dilakukan terhadap fungsi sign dan verify.

Test mencakup skenario normal dan beberapa edge case, seperti token dengan format salah, algoritma tidak didukung, public key yang salah, payload yang dimodifikasi, token kedaluwarsa, token belum aktif, dan klaim yang tidak sesuai. Dengan pengujian ini, library JWT dapat diuji secara terpisah dari aplikasi utama.

10. Bonus 3 : Docker

Aplikasi menyediakan konfigurasi Docker melalui file Dockerfile dan **docker-compose.yml**. Docker digunakan agar aplikasi dapat dijalankan secara konsisten pada lingkungan yang berbeda.

Dengan konfigurasi tersebut, aplikasi dapat dijalankan menggunakan perintah:

```
docker compose up --build
```

Docker Compose menjalankan aplikasi pada **port 3000** dan menggunakan volume untuk menyimpan data aplikasi. Dengan demikian, data pengguna dan pesan tetap tersimpan walaupun container dijalankan ulang.

Pengujian dan Analisis

Pengujian dilakukan untuk memastikan setiap fungsional berjalan dengan baik.

A. Uji autentikasi dan manajemen pengguna

Pengujian autentikasi dan manajemen pengguna dilakukan untuk memastikan bahwa pengguna dapat melakukan registrasi dan login dengan benar. Selain itu, pengujian ini juga memastikan bahwa sistem menolak data yang tidak valid, seperti email yang sudah terdaftar atau password yang salah.

1. Registrasi dengan data valid

Pada skenario ini, pengguna melakukan registrasi dengan email dan password baru. Registrasi dilakukan melalui antarmuka aplikasi.

Tabel 3.1 Skenario dan hasil pengujian registrasi dengan data valid

Parameter	Keterangan
Email	kripto@gmail.com
Password	kriptografi
Hasil	Registrasi Berhasil
Status	Berhasil

Gambar 1. Registrasi Kripto Chat

Setelah tombol registrasi ditekan, client membangkitkan pasangan kunci ECDH. Public key dikirim ke server, sedangkan private key dikirim dalam

bentuk terenkripsi sebagai `privateKeyEnvelope`. Server kemudian menyimpan data pengguna ke dalam basis data.



```

{
  "users": [
    {
      "id": "43d9bbd2-078a-454e-aeec-416dbfdbf97d",
      "email": "kripto@gmail.com",
      "passwordSalt": "1fiC2bCSJwi43AmCf9fHbA==",
      "passwordHash": "0rjNHR7vqJHGXS/xCrX3GdWpsKJUzr5J0MUc4zEEDUjwe367ZY+Md4HfnDejaunb3eLVE+YUFEyYm/f9Zqj",
      "publicKeyJwk": {
        "crv": "P-256",
        "ext": true,
        "key_ops": [],
        "kty": "EC",
        "x": "Kle4eZpSD0iLX42+CzT6GEwF8Ur-33R28vw64qeITLw",
        "y": "xdsg9YyavexjwLL_hwNHHjD3yPXVhdp0938KwIGVH8"
      },
      "privateKeyEnvelope": {
        "type": "pkcs8",
        "kdf": "PBKDF2",
        "hash": "SHA-256",
        "iterations": 150000,
        "aes": "AES-256-GCM",
        "salt": "zM2J5lIXmhG0UbK1n35LA==",
        "iv": "WFFPyhMIE8c0xfII",
        "ciphertext": "87eb0advp3T7X1y2hr3TW83tXsdJGhfnSnhPOE9nUq2Nw7j535PowASHZ5F4Kg5s7TyUV6RihH3dvp05BKzj"
      },
      "createdAt": "2026-05-12T03:18:38.138Z"
    }
  ]
}

```

Gambar 2. Data disimpan pada file JSON

Sistem berhasil membuat akun baru dan menyimpan data pengguna. *Password* tidak disimpan dalam bentuk plainteks, melainkan dalam bentuk *passwordhash* dan *passwordSalt*. Private key pengguna juga tidak disimpan secara langsung, tetapi disimpan dalam bentuk terenkripsi pada *privateKeyEnvelope*.

2. Registrasi dengan email terdaftar

Pada skenario ini, pengguna mencoba melakukan registrasi menggunakan email yang sebelumnya sudah pernah digunakan.

Tabel 3.2 Skenario dan hasil pengujian registrasi dengan data valid

Parameter	Keterangan
Email	kripto@gmail.com
Password	yes
Hasil	Server menolak registrasi
Status	Berhasil

Sistem berhasil mencegah duplikasi akun. Pemeriksaan email dilakukan sebelum data pengguna baru dimasukkan ke basis data, sehingga satu email hanya dapat digunakan oleh satu akun.

II4021 Kriptografi

Kripto Chat

Login Register

Email

kripto@gmail.com

Password

...

Register

Email is already registered

Gambar 3. Registrasi menggunakan email yang telah terdaftar

3. Login dengan password yang benar

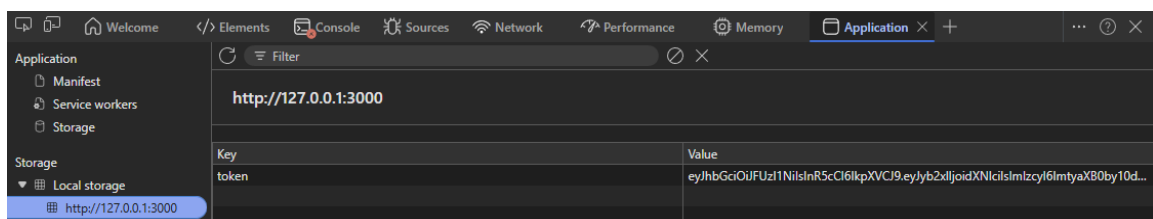
Pada skenario ini, pengguna melakukan login menggunakan email dan *password* yang sesuai dengan data registrasi.

Tabel 3.3 Skenario dan hasil pengujian registrasi dengan data valid

Parameter	Keterangan
Email	kripto@gmail.com
Password	kriptografi
Hasil	Login berhasil dan JWT diterbitkan
Status	Berhasil

Server mencari data pengguna berdasarkan email, lalu melakukan hashing ulang terhadap *password* yang dimasukkan menggunakan salt yang tersimpan. Jika hasil hash sesuai, server menerbitkan JWT dan mengirimkannya ke client.

Gambar 4. Login menggunakan password yang benar



Gambar 5. JWT tersimpan pada localStorage

Sistem berhasil memverifikasi *password* pengguna. Setelah *password* valid, server menerbitkan JWT yang digunakan sebagai bukti autentikasi pada endpoint berikutnya. Client juga dapat membuka halaman daftar kontak setelah login berhasil.

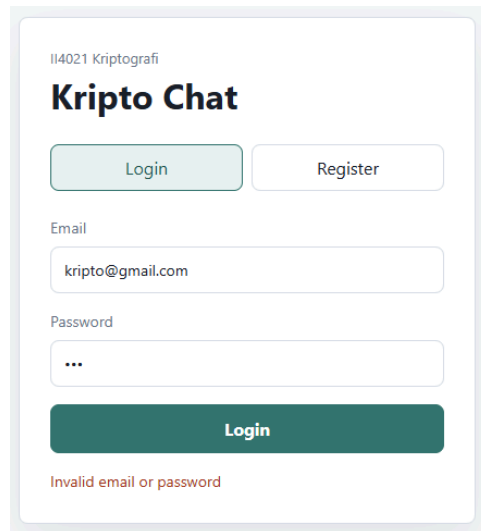
4. Login dengan password yang salah

Pada skenario ini, pengguna mencoba login menggunakan email yang terdaftar, tetapi *password* yang dimasukkan salah.

Tabel 3.4 Skenario dan hasil pengujian login dengan password salah

Parameter	Keterangan
Email	kripto@gmail.com
Password	kripto
Hasil	Login gagal

Status	Berhasil
--------	----------



The screenshot shows a web form titled "Kripto Chat" with the subtitle "II4021 Kriptografi". It has two buttons at the top: "Login" and "Register". Below these are input fields for "Email" (containing "kripto@gmail.com") and "Password" (containing three dots). A large green "Login" button is at the bottom. Below the button, a red error message reads "Invalid email or password".

Gambar 6. Login menggunakan password yang salah

Sistem berhasil menolak login karena hash *password* yang dihitung dari input pengguna tidak sama dengan hash *password* yang tersimpan. Karena autentikasi gagal, server tidak menerbitkan JWT.

B. Uji implementasi JSON Web Token (JWT)

Pengujian implementasi JSON Web Token dilakukan untuk memastikan bahwa library JWT yang dibuat dapat menghasilkan token, memverifikasi token, membaca payload, serta menolak token yang tidak valid. Pengujian ini mencakup penggunaan public key yang benar, public key yang salah, token dengan format tidak valid, serta token dengan klaim waktu yang tidak sesuai.

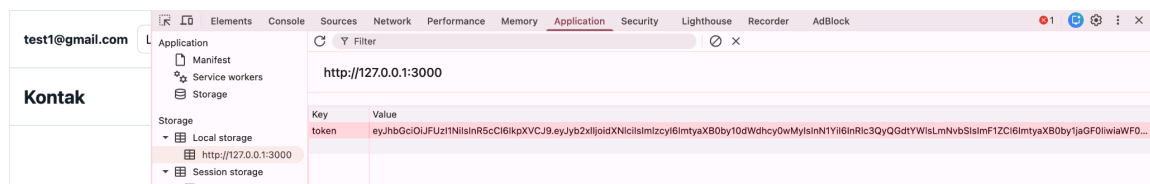
1. Verify dengan Public Key Benar

Pada skenario ini, fungsi `sign` digunakan untuk menghasilkan JWT menggunakan private key. Token yang dihasilkan kemudian diverifikasi menggunakan fungsi `verify` dengan public key yang sesuai. Jika public key yang digunakan merupakan pasangan dari private key, maka token akan dinyatakan valid.

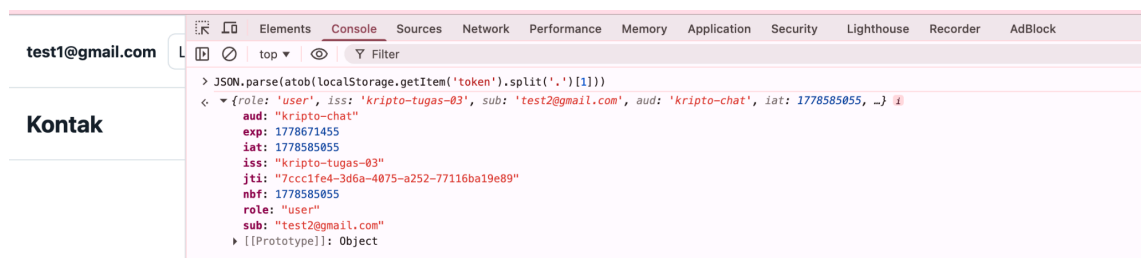
Setelah proses login berhasil, JWT disimpan pada `localStorage` browser. Token tersebut kemudian dibaca melalui browser console. Payload JWT juga dapat dibaca dengan melakukan `decode` pada bagian kedua token, yaitu bagian payload.

Tabel 3.5 Skenario dan hasil pengujian login dengan password salah

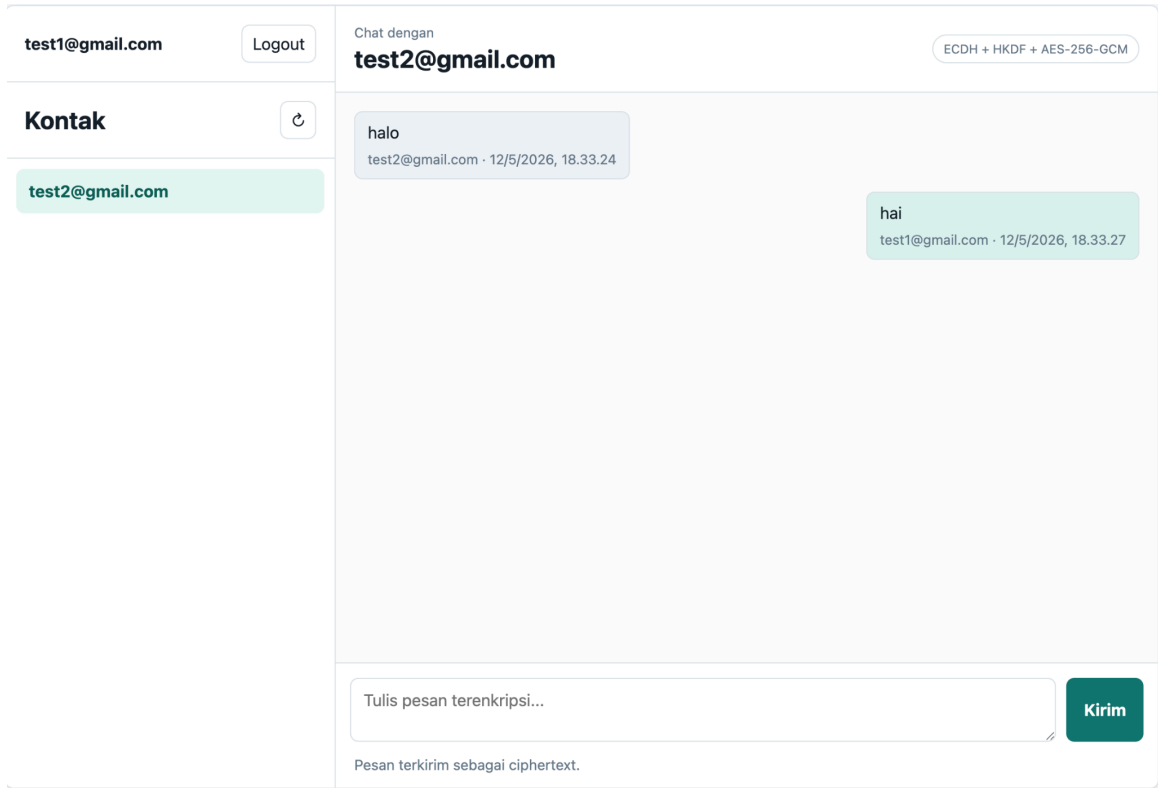
Parameter	Keterangan
Private key	MIGHAgEAMBMGBYqGSM49AgEGCCqGSM49AwEHBG0wawIB AQQgBcdmWd97sEswufHN zgKqmaPRq9n0uuW4qysvFY7b/96hRANCAARN7FL+EaVBVwM TFqWUSHTJwfKWkOEP PHXCILGqtIgFib+fTrG/+Qe8AQ9tHQbV3JSiJEoQ5OuUh7A+8ROe QBwK
Public key	MFkwEwYHKoZIzj0CAQYIKoZIzj0DAQcDQgAETexS/hGIQVcDExa lIEh7ScHyIpDh Dzx1wiCxqrSIBSG/n06xv/kHvAEPbR0G1dyUoiRKEOTrllewPvETnk AcCg==
Token	eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJyb2xlIjoiaXNlciIsImIz cyI6ImtyaXB0by10dWdhcy0wMyIsInN1YiI6InRlc3QyQWdtYWwzLmNvbSIsImF1ZCI6ImtyaXB0by1jaGF0IiwiaWF0IjoxNzc4NTg1MD U1LCJuaWYiOiJlZ3NzZg1ODUwNTUsImV4cCI6MTc3ODY3MTQ1NSw ianRpljoiN2NjYzFmZTQtM2Q2YS00MDc1LWEyNTItNzc4MTZiYTE 5ZTg5In0.ppUaOpmgROlf9izqrMCv1FUpEpFH8O4-Q70v89LgIQ R4xAuiCyEIEB4bYwMS6zntw1m4DdL5QwZm36P-qCdekQ



Gambar 7. JWT hasil login yang tersimpan pada localStorage



Gambar 8. Payload JWT berhasil dibaca melalui browser console



Gambar 9. Tampilan pesan dengan public key yang benar

Token dinyatakan valid karena signature JWT berhasil diverifikasi menggunakan public key yang sesuai dengan private key saat token dibuat. Payload dapat dibaca karena JWT menggunakan format JWS, sehingga bagian payload hanya dikodekan dalam Base64URL dan bukan dienkripsi.

2. **Verify dengan public key yang salah**

Pada skenario ini, pengujian dilakukan dengan mengganti public key yang digunakan untuk memverifikasi JWT. Token tetap dihasilkan menggunakan private key asli, tetapi proses verifikasi dilakukan menggunakan public key lain yang tidak sesuai.

Tabel 3.6 Skenario dan hasil pengujian login dengan password salah

Parameter	Keterangan
Private key	MIGHAgEAMBMGBByqGSM49AgEGCCqGSM49AwEHBG0wawIB AQQgBcdmWD97sEswufHN zgKqmaPRq9n0uuW4qysvFY7b/96hRANCAARN7FL+EaVBVwM TFqWUShtJwfKWkOEP PHXCILGqtIgFib+fTrG/+Qe8AQ9tHQbV3JSiJEoQ5OuUh7A+8ROe QBwK

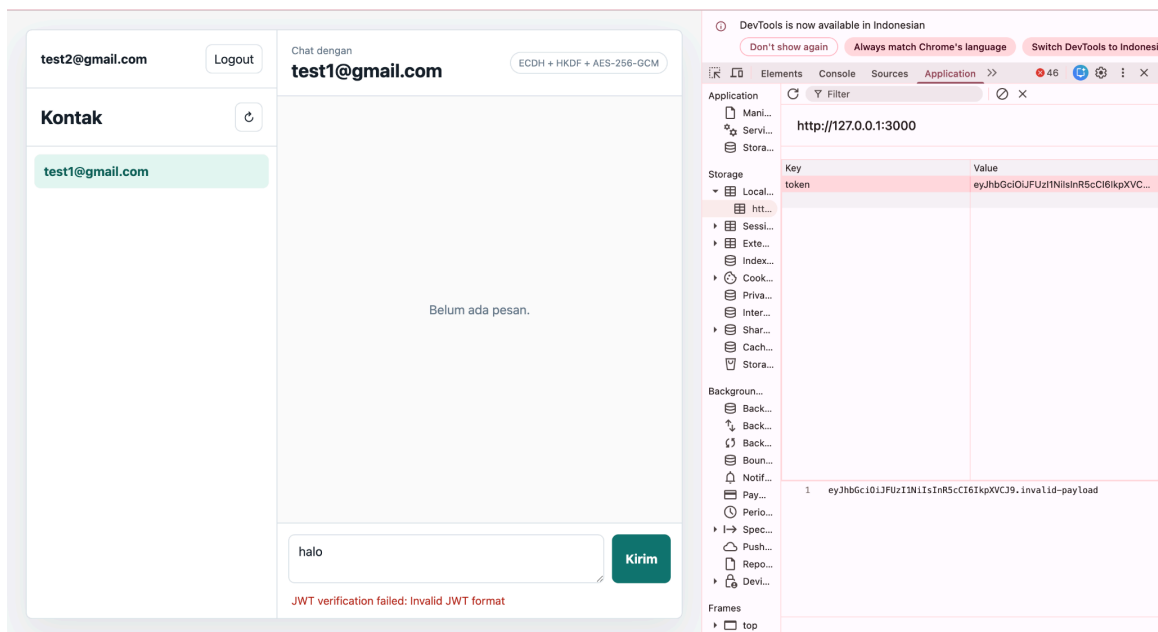
header.payload.signature

Token yang tidak memiliki format tersebut dianggap sebagai token tidak valid.

Tabel 3.7 Skenario pengujian token dengan format tidak valid

Parameter	Keterangan
Token	eyJhbGciOiJFUzI1NiIsInR5cCI6IkpXVCJ9.invalid-payload

Gambar 12. Hasil verifikasi JWT gagal dengan format tidak valid



Gambar 13. Tampilan pesan saat format token invalid

4. Uji token dengan klaim waktu (exp/nbf) yang tidak sesuai

Pada skenario ini, pengujian dilakukan terhadap klaim waktu pada JWT. Klaim **exp** digunakan untuk menentukan waktu kedaluwarsa token, sedangkan klaim **nbf** digunakan untuk menentukan waktu mulai token boleh digunakan.

Pengujian dilakukan dengan dua kondisi. Pertama, token dibuat dengan nilai exp yang sudah lewat, sehingga token dianggap kedaluwarsa. Kedua, token dibuat dengan nilai nbf di masa depan, sehingga token belum boleh digunakan pada waktu pengujian.

Tabel 3.8 Skenario pengujian exp dan nbf

Parameter	Nilai Normal	Nilai Pengujian	Keterangan
exp	now + 24 * 60 * 60	now - 60	Token dibuat sudah kedaluwarsa 60 detik sebelum waktu saat ini.
nbf	now	now + 3600	Token baru boleh digunakan 1 jam setelah waktu saat ini.

```

claims: {
  iss: 'kripto-tugas-03',
  sub: user.email,
  aud: 'kripto-chat',
  iat: now,
  nbf: now,
  exp: now - 60,
  jti: crypto.randomUUID()
},

```

Kripto chat running at http://127.0.0.1:3000
[auth] JWT verification failed: JWT expired
[auth] JWT verification failed: JWT expired

Gambar 14. Hasil verifikasi JWT gagal dengan exp yang kedaluwarsa

Tulis pesan terenkripsi...

Kirim

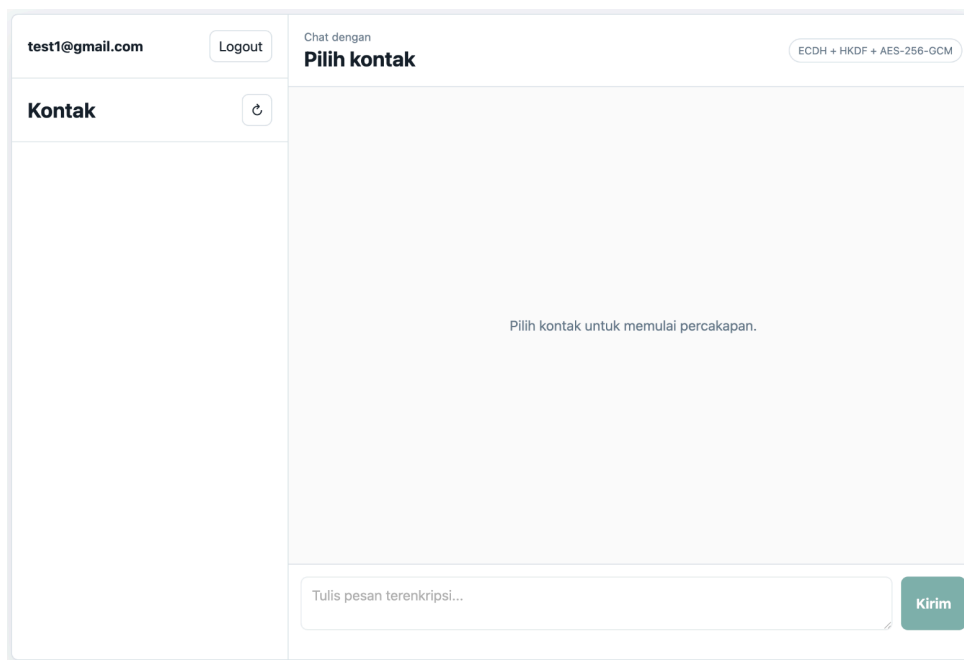
JWT verification failed: JWT expired

Gambar 15. Tampilan pesan saat exp yang kedaluwarsa


```
iss: 'kripto-tugas-03',
sub: user.email,
aud: 'kripto-chat',
iat: now,
nbf: now + 3600,
exp: now + 24 * 60 * 60,
jti: crypto.randomUUID()
},
```

Kripto chat running at http://127.0.0.1:3000
[auth] JWT verification failed: JWT not active yet

Gambar 16. Hasil verifikasi JWT gagal dengan nbf belum aktif



Gambar 17. Tampilan pesan saat nbf belum aktif

Hasil pengujian menunjukkan bahwa sistem berhasil memvalidasi klaim waktu pada JWT. Ketika nilai **exp** dibuat lebih kecil dari waktu saat ini, token dianggap kedaluwarsa dan request ditolak oleh server. Ketika nilai **nbf** dibuat lebih besar dari waktu saat ini, token dianggap belum aktif dan request juga ditolak. Dengan demikian, token hanya dapat digunakan selama berada dalam rentang waktu yang valid. Pada aplikasi utama, opsi **ignoreExp** dan **ignoreNbf** tidak digunakan, sehingga token dengan klaim waktu yang tidak sesuai tetap ditolak oleh sistem.

C. Uji pembentukan kunci komunikasi ECDH dan KDF

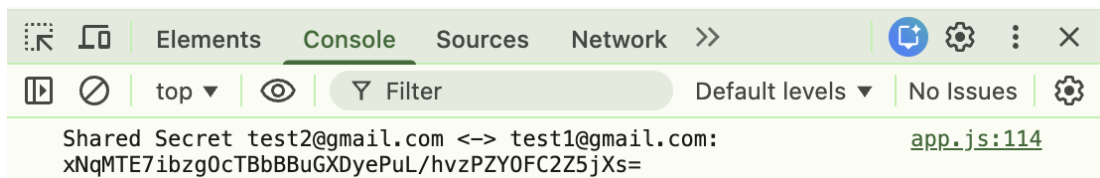
Pengujian pembentukan kunci komunikasi dilakukan untuk memastikan bahwa dua pengguna dapat menghasilkan kunci komunikasi yang sama menggunakan ECDH dan KDF. Kunci tersebut kemudian digunakan untuk mengenkripsi dan mendekripsi pesan.

1. Dua pengguna melakukan *key exchange*

Pada skenario ini, dua pengguna melakukan percakapan. Masing-masing pengguna memiliki pasangan kunci ECDH. Ketika salah satu pengguna memilih kontak, client menggunakan private key miliknya sendiri dan public key kontak untuk menghasilkan shared secret.

Tabel 3.9 Skenario dan hasil pengujian login dengan password salah

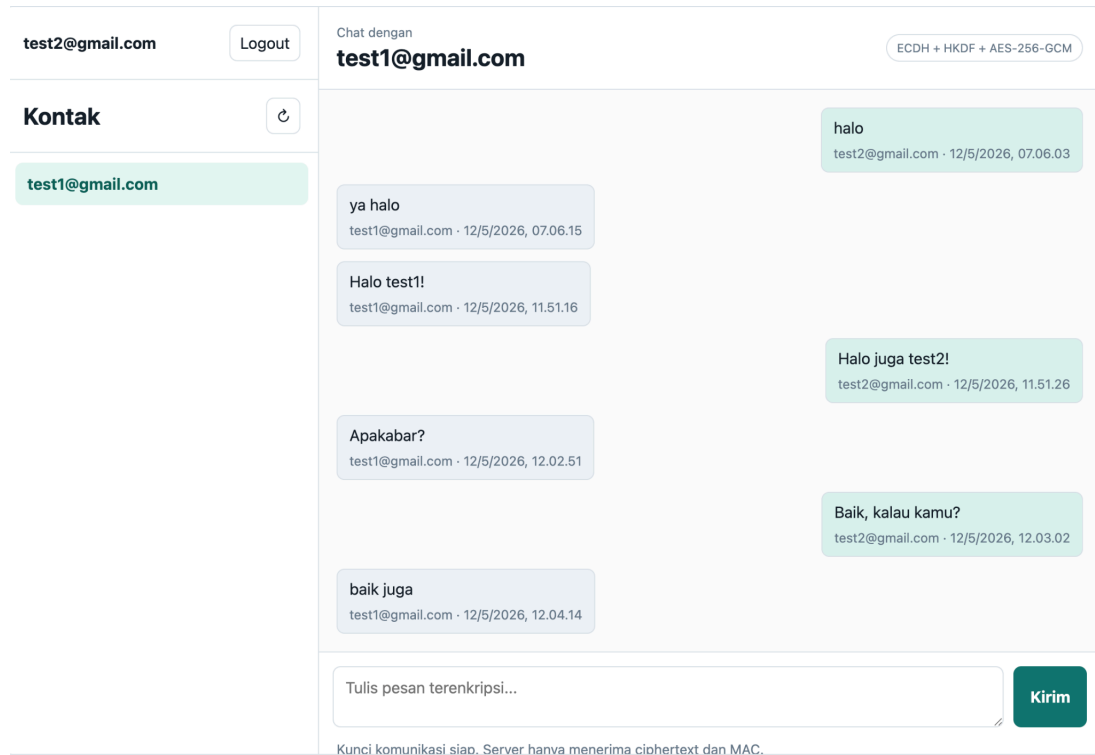
Parameter	Keterangan
User Pertama	test1@gmail.com
User Kedua	test2@gmail.com
Algoritma	Login gagal
Hasil	Pesan dapat didekripsi oleh penerima
Status	Berhasil



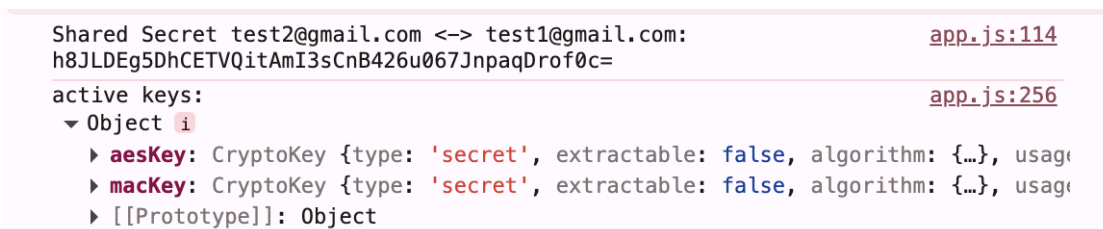
Gambar 18. Shared Secret pada kedua pengguna

2. *Shared secret* berhasil menjadi kunci simetris

Pada skenario ini, shared secret hasil ECDH diproses menggunakan KDF untuk menghasilkan kunci simetris. Kunci simetris tersebut digunakan untuk proses enkripsi dan dekripsi pesan.



Gambar 19. Pesan hasil dari enkripsi dan dekripsi



Gambar 20. Pesan terenkripsi disimpan pada basis data

Pada console terlihat bahwa proses ECDH menghasilkan shared secret. Setelah itu, proses HKDF menghasilkan objek **activeKeys** yang berisi **aesKey** dan **macKey**. Kunci tersebut ditampilkan sebagai **CryptoKey** dengan sifat **extractable: false**, sehingga nilai mentah kunci tidak dapat dibaca langsung. Hal ini menunjukkan bahwa shared secret berhasil diturunkan menjadi kunci simetris yang digunakan untuk enkripsi pesan dan autentikasi MAC.

D. Uji enkripsi dan dekripsi pesan

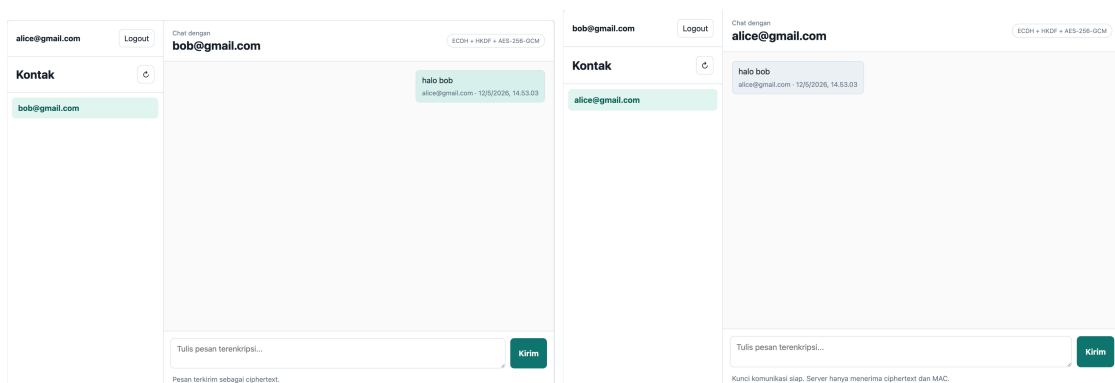
Pengujian enkripsi dan dekripsi pesan dilakukan untuk memastikan bahwa pesan tidak dikirim dan disimpan dalam bentuk plainteks. Pesan yang diketik oleh pengguna akan dienkripsi terlebih dahulu menggunakan kunci AES hasil derivasi HKDF. Setelah diterima oleh pengguna tujuan, pesan akan didekripsi kembali menggunakan kunci yang sama.

1. Kirim pesan dengan kunci yang benar

Pada skenario ini, pengguna mengirim pesan kepada kontak setelah kunci komunikasi berhasil terbentuk. Pesan dikirim menggunakan kunci yang benar, sehingga penerima dapat melakukan dekripsi dan membaca pesan dalam bentuk plainteks.

Tabel 3.10 Skenario dan hasil pengujian pengiriman pesan dengan kunci yang benar

Parameter	Keterangan
User Pertama	alice@gmail.com
User Kedua	bob@gmail.com
Uji coba	Kunci benar
Hasil	Pesan dapat ter enkripsi & dekripsi oleh penerima
Status	Berhasil



Gambar 21. Alice mengirim pesan kepada Bob dan pesan terkirim ke Bob

```

"messages": [
  {
    "id": "20e777ab-bf83-415c-ab5c-0425c3fc2f8e",
    "senderEmail": "alice@gmail.com",
    "receiverEmail": "bob@gmail.com",
    "ciphertext": "7mRYKZGANU2RbieU+XFYZ+G5Hp3Pjrf4",
    "iv": "lxCKNB6UecshpPqr",
    "mac": "Nlum0NmjK6/bhHUKD99LEWx96kQxzaG7iMcT9DQMoik=",
    "timestamp": "2026-05-12T07:53:03.214Z",
    "alg": "AES-256-GCM+HMAC-SHA-256"
  }
]

```

Gambar 22. Pesan Alice kepada Bob dienkripsi pada basis data

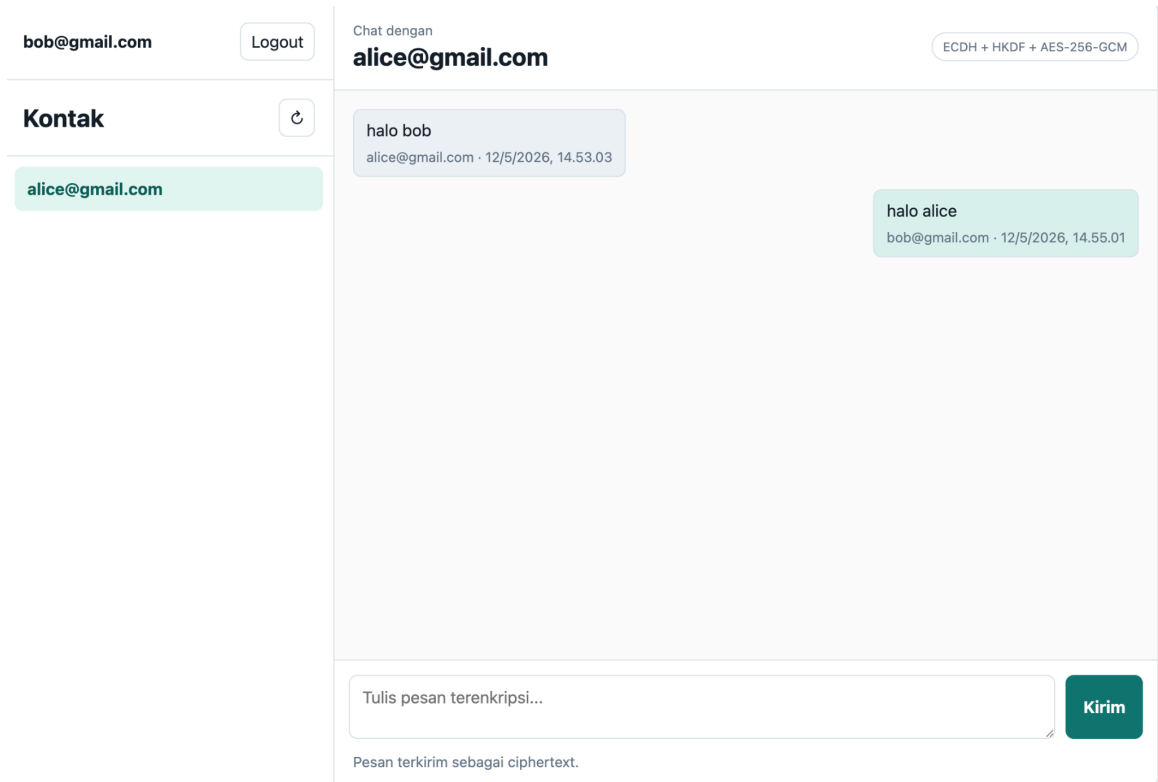
Hasil pengujian menunjukkan bahwa pesan yang dikirim oleh pengguna dapat diterima dan ditampilkan pada sisi penerima sebagai teks biasa. Sementara itu, pada file basis data, pesan tidak tersimpan sebagai plainteks, melainkan tersimpan dalam field ciphertext.

2. Kirim pesan dengan kunci yang salah atau tidak sesuai

Pada skenario ini, pengujian dilakukan dengan mengubah data pesan, seperti ciphertext atau IV, sehingga data yang diterima tidak lagi sesuai dengan data asli. Perubahan ini menyebabkan proses dekripsi gagal.

Tabel 3.11 Skenario dan hasil pengujian pengiriman pesan dengan kunci yang salah

Parameter	Keterangan
User Pertama	bob@gmail.com
User Kedua	alice@gmail.com
Uji coba	Data tidak sesuai
Hasil	Pesan tidak dapat di dekripsi oleh penerima
Status	Berhasil



Gambar 23. Bob mengirim pesan kepada Alice

```
{
  "id": "f7a1265c-d15c-445e-81f8-1da7edc7af27",
  "senderEmail": "bob@gmail.com",
  "receiverEmail": "alice@gmail.com",
  "ciphertext": "TiFJ4aPLhpg4Sns9Z5IBeIVFAAQza5mN14=",
  "iv": "8Q7dy4GybSk2stVr",
  "mac": "CP1qp8vKnhR2DyAxLfb02eVyEAmCnwu60EZjmNsIvGg=",
  "timestamp": "2026-05-12T07:55:01.564Z",
  "alg": "AES-256-GCM+HMAC-SHA-256"
}
```

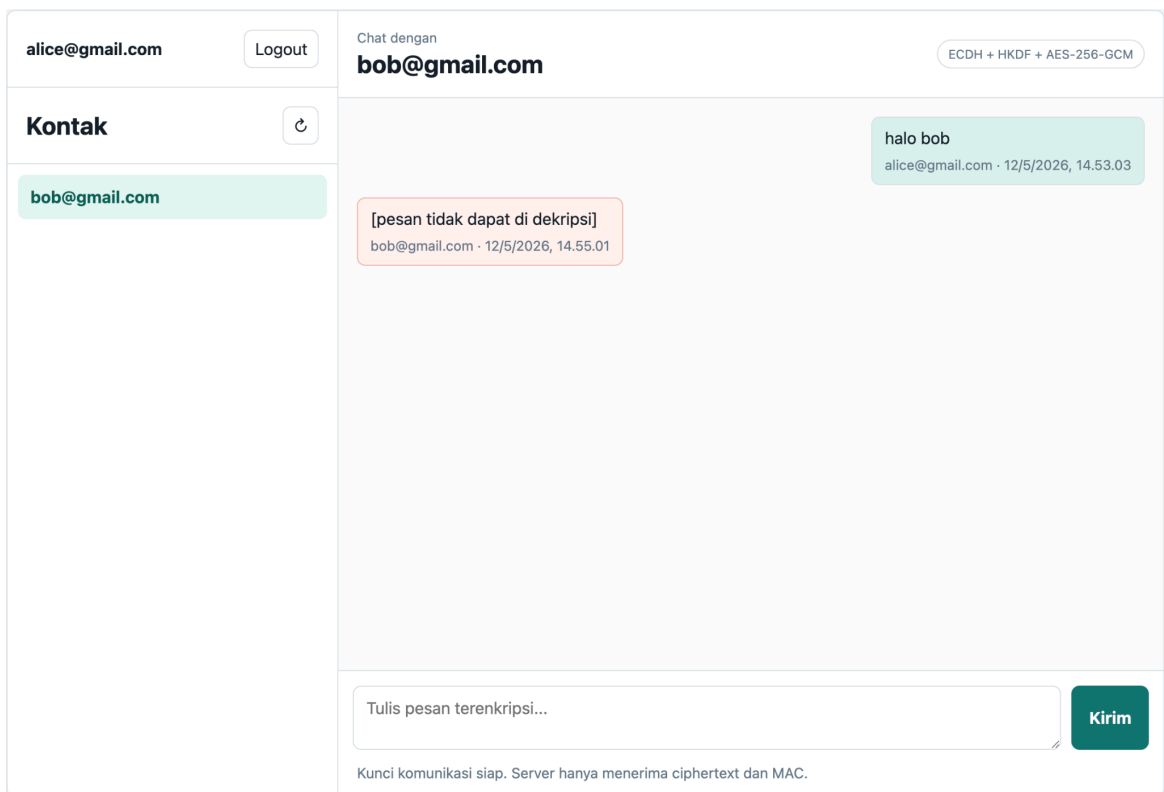
Gambar 24. Pesan Bob yang telah dienkripsi

```

{
  "id": "f7a1265c-d15c-445e-81f8-1da7edc7af27",
  "senderEmail": "bob@gmail.com",
  "receiverEmail": "alice@gmail.com",
  "ciphertext": "TiFJ4aPLhpg4Sns9Z5IBEaIVFAAQza5mN2x=",
  "iv": "8Q7dy4GybSk2stVr",
  "mac": "CP1qp8vKnhR2DyAxLfb02eVyEAmCnwu60EZjmNsIvGg=",
  "timestamp": "2026-05-12T07:55:01.564Z",
  "alg": "AES-256-GCM+HMAC-SHA-256"
}

```

Gambar 25. Modifikasi pada ciphertexts pesan Bob



Gambar 26. Ciphertexts yang dimodifikasi tidak dapat didekripsi

Hasil pengujian menunjukkan bahwa ketika data pesan tidak sesuai, aplikasi tidak dapat mendekripsi pesan dengan benar. Sistem kemudian menampilkan penanda error pada antarmuka.

E. Uji integritas dan autentikasi pesan (MAC):

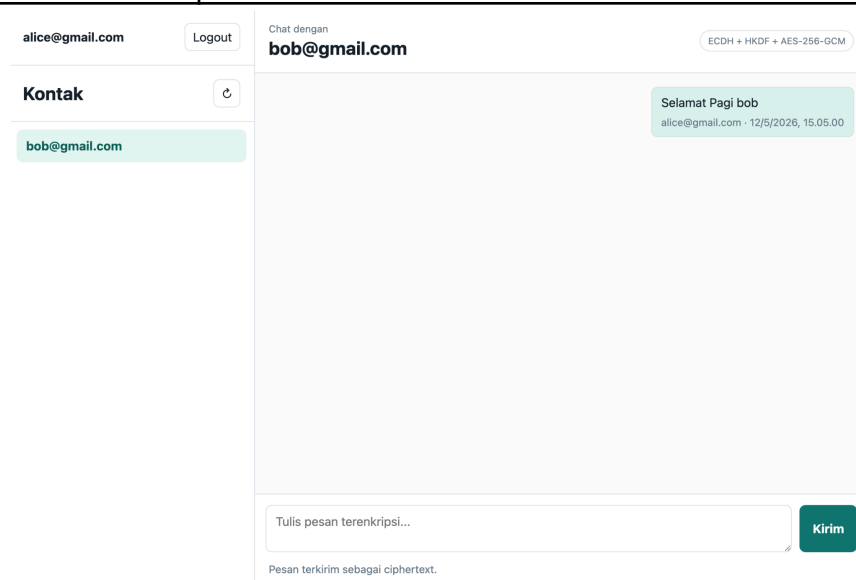
Pengujian MAC dilakukan untuk memastikan bahwa aplikasi dapat mendeteksi perubahan pada pesan sebelum pesan diproses lebih lanjut. MAC dibuat menggunakan HMAC-SHA-256 dari metadata pesan dan ciphertext. Nilai MAC ini dikirim bersama pesan dan diverifikasi kembali oleh penerima.

1. Kirim pesan dengan MAC yang valid

Pada skenario ini, pesan dikirim tanpa perubahan pada ciphertext, IV, timestamp, pengirim, penerima, maupun MAC. Dengan kondisi tersebut, nilai MAC yang dihitung ulang oleh penerima akan sama dengan MAC yang dikirim oleh pengirim.

Tabel 3.12 Skenario dan hasil pengujian pengiriman pesan dengan autentikasi pesan yang benar

Parameter	Keterangan
User Pertama	alice@gmail.com
User Kedua	bob@gmail.com
Uji coba	MAC benar
Hasil	Pesan dapat diterima oleh penerima
Status	Berhasil



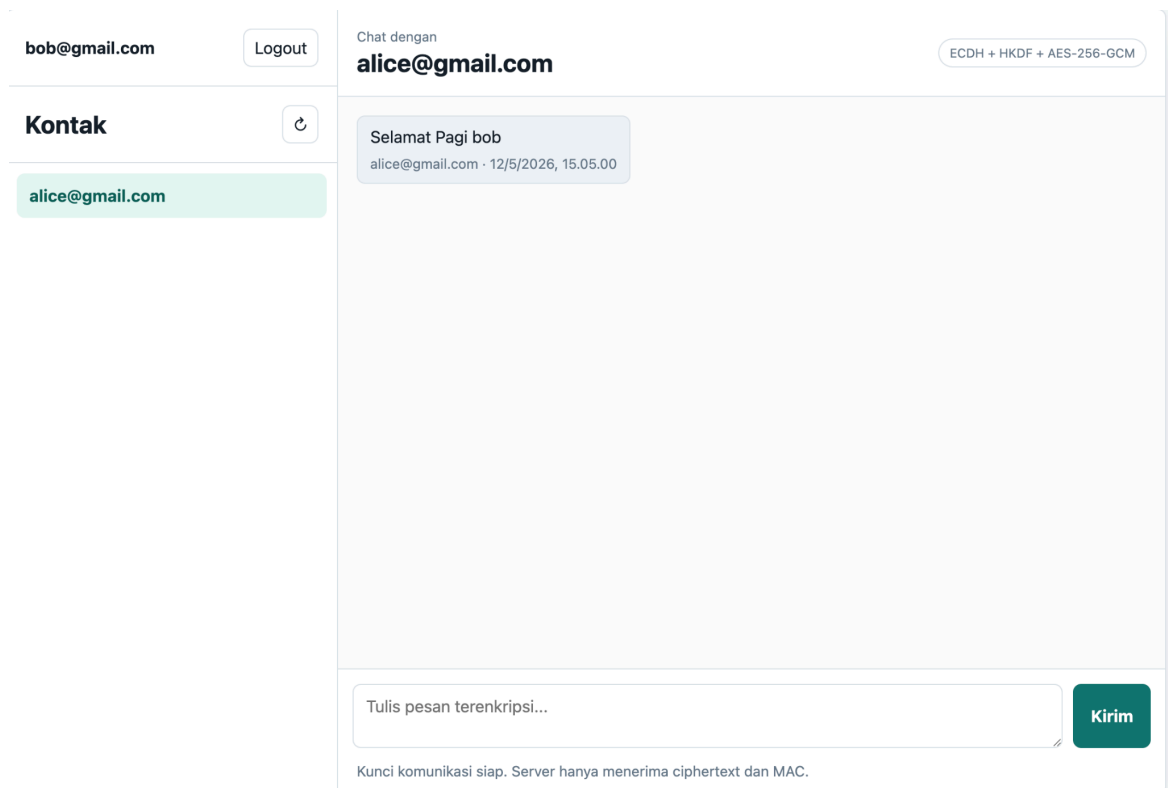
Gambar 27. Pesan Alice kepada Bob


```

"messages": [
  {
    "id": "225f055e-6a89-45d5-a61b-675c93739a1b",
    "senderEmail": "alice@gmail.com",
    "receiverEmail": "bob@gmail.com",
    "ciphertext": "uxKwP5w7Ck3HEE6zDuWypCdwcJD/xLxF8qRBQECLgvc=",
    "iv": "RRQ3HaSUHKHQCQhE",
    "mac": "aiy68mJ0eLSeNofYVkyCh9EyqUQ0wZz4zKiKiwDmYh0=",
    "timestamp": "2026-05-12T08:05:00.712Z",
    "alg": "AES-256-GCM+HMAC-SHA-256"
  }
]

```

Gambar 28. MAC yang valid



Gambar 29. Pesan Alice terkirim kepada Bob

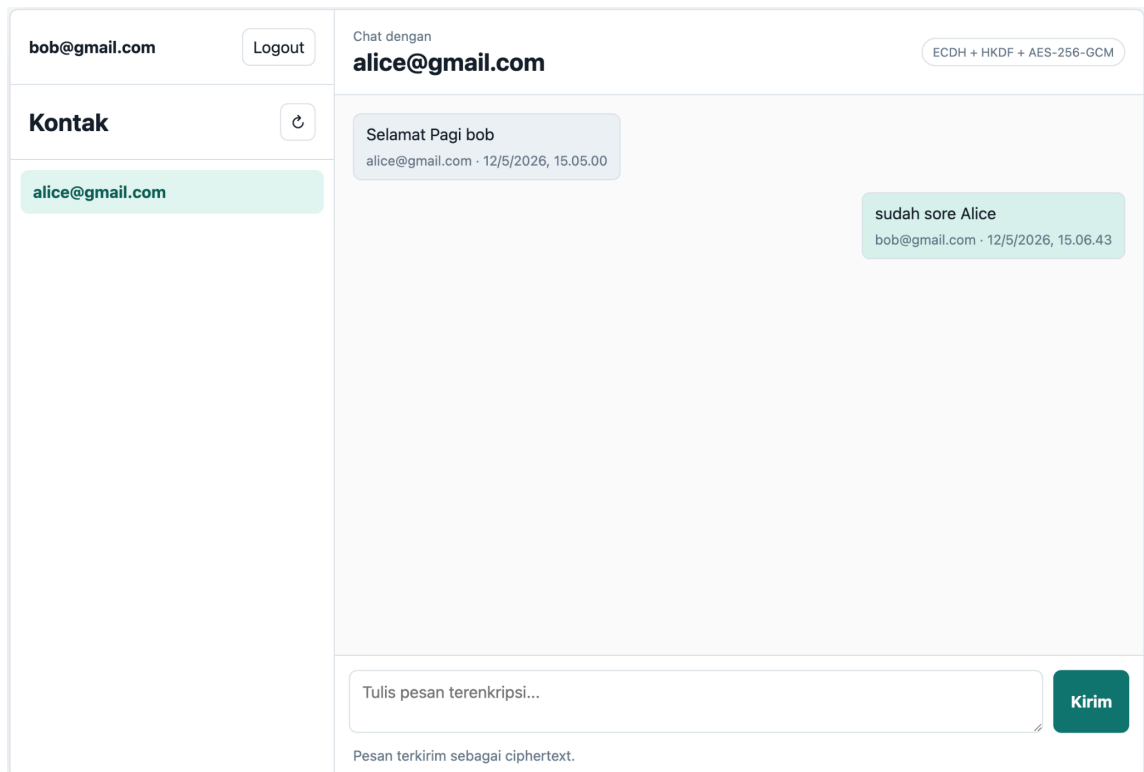
Hasil pengujian menunjukkan bahwa pesan dengan MAC valid dapat diterima, diverifikasi, dan didekripsi dengan benar. Pesan kemudian ditampilkan pada antarmuka sebagai plaintexts.

2. Pesan dengan MAC Tidak Valid

Pada skenario ini, sebagian isi pesan atau nilai MAC diubah secara manual pada basis data. Setelah itu, penerima membuka kembali percakapan untuk memeriksa apakah pesan masih dianggap valid.

Tabel 3.13 Skenario dan hasil pengujian login dengan autentikasi pesan yang benar

Parameter	Keterangan
User Pertama	bob@gmail.com
User Kedua	alice@gmail.com
Uji coba	MAC diubah
Hasil	Pesan tidak dapat diproses oleh penerima
Status	Berhasil



Gambar 30. Pesan Bob kepada Alice

```

{
  "id": "a2bfecad-cb2c-4fbb-aacf-7bd1469e9340",
  "senderEmail": "bob@gmail.com",
  "receiverEmail": "alice@gmail.com",
  "ciphertext": "fQy50gzsmgPRvBhmu077sfe/kRij9snnXtKq8SktUis=",
  "iv": "yg8XRpIX+qj/SyYX",
  "mac": "WJDgkIDY95hUEX0x2Exr0WtDB5zZR/sMh1zB99JqA8U=",
  "timestamp": "2026-05-12T08:06:43.812Z",
  "alg": "AES-256-GCM+HMAC-SHA-256"
}

```

Gambar 31. MAC pesan yang valid dari bob

```

{
  "id": "a2bfecad-cb2c-4fbb-aacf-7bd1469e9340",
  "senderEmail": "bob@gmail.com",
  "receiverEmail": "alice@gmail.com",
  "ciphertext": "fQy50gzsmgPRvBhmu077sfe/kRij9snnXtKq8SktUis=",
  "iv": "yg8XRpIX+qj/SyYX",
  "mac": "WJDgkIDY95hUEX0x2Exr0WtDB5zZR/sMh1zB99JqAXD=",
  "timestamp": "2026-05-12T08:06:43.812Z",
  "alg": "AES-256-GCM+HMAC-SHA-256"
}

```

Gambar 32. Modifikasi MAC dari pesan

alice@gmail.com
Logout

Kontak

bob@gmail.com

Chat dengan bob@gmail.com
ECDH + HKDF + AES-256-GCM

Selamat Pagi bob
alice@gmail.com · 12/5/2026, 15.05.00

[MAC tidak valid]
bob@gmail.com · 12/5/2026, 15.06.43

Tulis pesan terenkripsi...

Kirim

Kunci komunikasi siap. Server hanya menerima ciphertext dan MAC.

Gambar 33. Pesan Bob yang diterima Alice menggunakan MAC yang tidak valid

Hasil pengujian menunjukkan bahwa setelah nilai MAC atau isi pesan diubah, aplikasi menampilkan penanda [MAC tidak valid]. Pesan tidak diproses sebagai pesan sah karena nilai MAC tidak cocok dengan hasil perhitungan ulang pada sisi penerima.

Kesimpulan

Berdasarkan hasil implementasi dan pengujian yang telah dilakukan, kami dapat menarik beberapa kesimpulan sebagai berikut:

1. Autentikasi Pengguna Berhasil Diimplementasikan

Pada aplikasi Kripto Chat, *server* melakukan autentikasi pengguna menggunakan JSON Web Token yang dilengkapi dengan tanda tangan digital *Elliptic Curve Digital Signature Algorithm* beserta algoritma ES256, ES384, dan ES512. Proses autentikasi ini dilengkapi dengan fungsi *sign* dan *verify* dan penggunaan *unit test* untuk memastikan autentikasi berjalan baik.

2. Keamanan *Password* Terjamin

Password pengguna yang diterima oleh *server* telah melalui proses *hashing* dan dikombinasikan dengan *salt* sebelum disimpan ke basis data. Dengan demikian *server* tidak mengetahui *password* sebenarnya milik pengguna. Hal ini terbukti dengan *password* yang tersimpan pada *file db.json* (basis data) berbeda dengan *password* yang dimasukkan pengguna.

3. Integritas Pesan Terjamin

Setiap pesan yang dikirim dan diterima telah dilindungi HMAC-SHA-256, oleh karena itu perubahan sekecil apapun terhadap pesan oleh pihak luar dapat diketahui dan bahkan tidak terkirim (muncul teks 'MAC tidak valid').

4. Enkripsi Pesan End-To-End Tercapai

Pesan yang dikirimkan telah dienkripsi menggunakan AES-256-CBC pada *client* sebelum diterima oleh *server*. *Server* berperan sebagai penerima dan penerus pesan yang berupa cipherteks tanpa mengetahui isi, kunci maupun konteks dari pesan yang dikirimkannya.

5. Pembentukan *Shared Secret* Berhasil

Kedua pihak berhasil membentuk *shared secret* menggunakan ECDH P-256 pada *client*. *Shared secret* kemudian diturunkan menggunakan HKDF SHA-256 yang menghasilkan kunci AES-256 untuk proses enkripsi pesan.

6. Implementasi Berhasil Memenuhi Spesifikasi dan Ketentuan Program

Berdasarkan pengujian yang telah dilakukan pada bagian 'Pengujian dan Analisis' dapat disimpulkan bahwa aplikasi Kripto Chat telah memenuhi spesifikasi dan ketentuan program.

Daftar Pustaka

Munir, R. (2026). *25 ECC STI Bagian 1 dan 2 2026* [Materi kuliah II4021 Kriptografi]. Program Studi Sistem dan Teknologi Informasi, Sekolah Teknik Elektro dan Informatika, Institut Teknologi Bandung.

Munir, R. (2026). *26 Fungsi hash kriptografi* [Materi kuliah II4021 Kriptografi]. Program Studi Sistem dan Teknologi Informasi, Sekolah Teknik Elektro dan Informatika, Institut Teknologi Bandung.

Jones, M., Bradley, J., & Sakimura, N. (2015). *JSON Web Token (JWT)*. Internet Engineering Task Force. <https://www.rfc-editor.org/rfc/rfc7519>

jsonwebtoken (npm): Furneaux, N. (2015). jsonwebtoken (npm package). <https://www.npmjs.com/package/jsonwebtoken>

Web Crypto API: Mozilla Developer Network. (2024). Web Crypto API. MDN Web Docs. https://developer.mozilla.org/en-US/docs/Web/API/Web_Crypto_API

Lampiran

Pranala Repositori

[Link](#)

Pranala Video Demo

[Link](#)

Pembagian Tugas

Nama	NIM	Tugas
Willhelmina Rachel Silalahi	18222049	Dokumen, BE
Natahaniel Liady	18222114	Dokumen, BE + FE
Daniel A. M. Sipayung	18223031	Dokumen, FE, dan implementasi JWT